Yakup Koray Budanaz

SoftHier Meeting October 13



Implemented Multiple Components of the Explicit Vectorization Pipeline

- Explicit vectorization is mostly completed. Couple edges cases remaining
- Currently:
  - Implementing support for the remaining edge cases
  - Extending LoopToMap to catch all-paralellizable loops of CloudSC
  - Applying the vectorization pass on CloudSC (initial tests will be on the CPU)

Branch Fusion Pass

- Branches within maps are not suitable for SIMD instructions
- However most branching is SIMT-suitable where write-sets of branches are the same and threads do the same / similar computation and the threads are activated through predicates:

```
1  if a[i, j] > 0.5:
2      c[i, j] = a[i, j] * b[i, j]
3      d[i, j] = 1 - c[i, j]
4  else:
5      c[i, j] = 0.0
6      d[i, j] = 0.0
7
```

SIMT

Branch Fusion Pass

- Branches within maps are not suitable for SIMD instructions
- However most branching is SIMT-suitable where write-sets of branches are the same and threads do the same / similar computation and the threads are activated through predicates:

- SIMD-suitable transformation is possible for the patterns arising in CloudSC:

```
1  if a[i, j] > 0.5:
2      c[i, j] = a[i, j] * b[i, j]
3      d[i, j] = 1 - c[i, j]
4  else:
5      c[i, j] = 0.0
6      d[i, j] = 0.0
7
```

SIMT

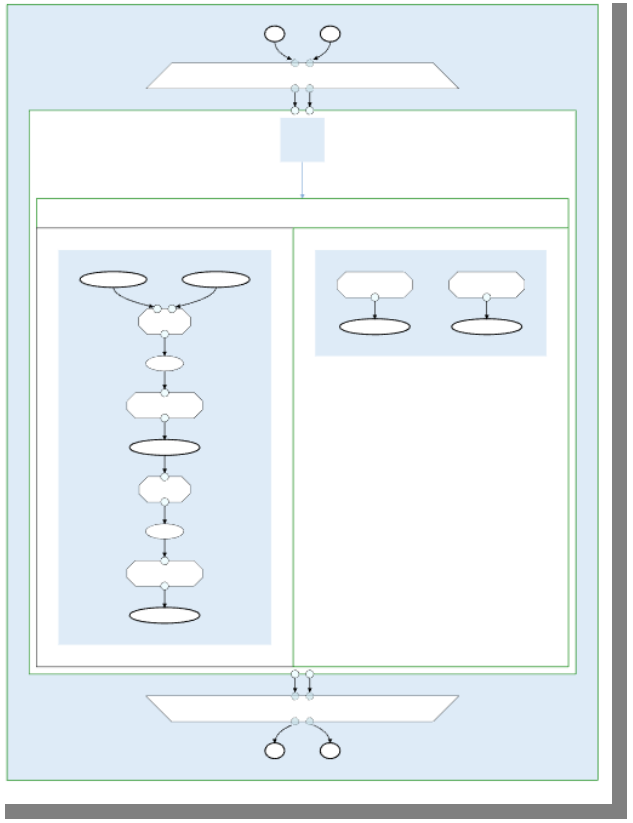
```
1  f_cond = 1.0 if (a[i, j] > 0.5) else 0.0
2  c[i, j] = (a[i, j] * b[i, j]) * f_cond
3            + (0.0) * (1.0 - f_cond)
4  d[i, j] = (1.0 - c[i, j]) * f_cond
5            + (0.0) * (1.0 - f_cond)
```

SIMD

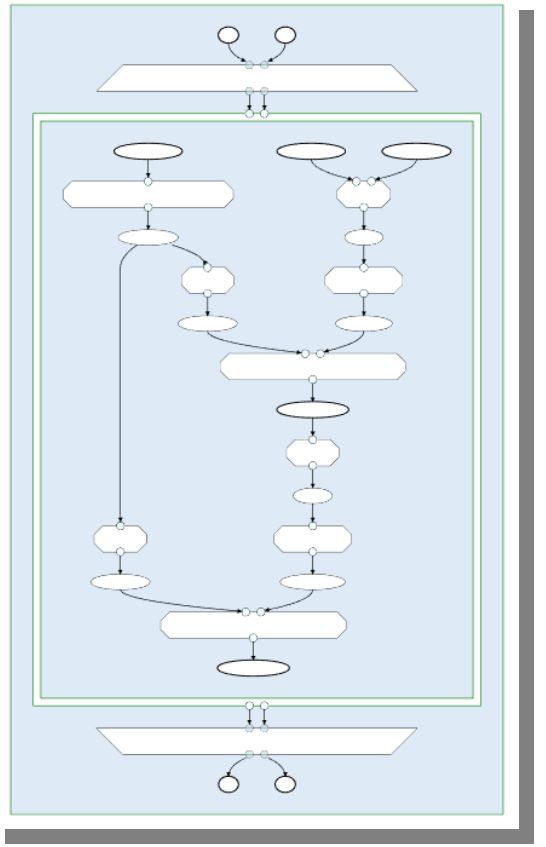
The transformation may result in increase of total FLOPs / bytes transferred, but enables vectorization.

Branch Fusion

```
1  if a[i, j] > 0.5:
2      c[i, j] = a[i, j] * b[i, j]
3      d[i, j] = 1 - c[i, j]
4  else:
5      c[i, j] = 0.0
6      d[i, j] = 0.0
7
```



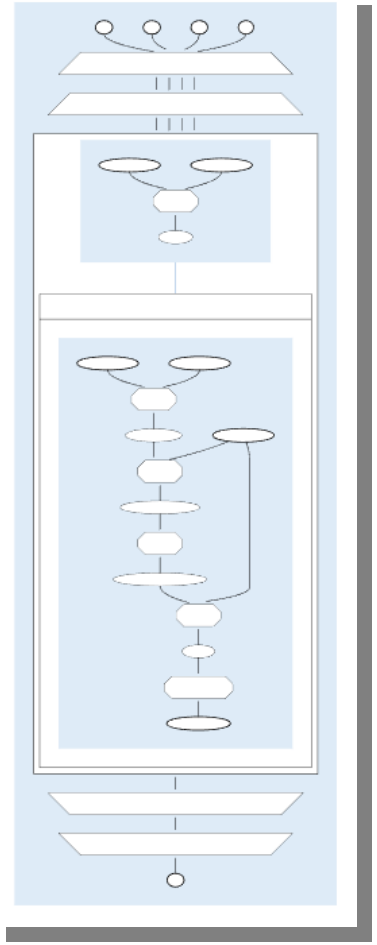
BranchFusion()



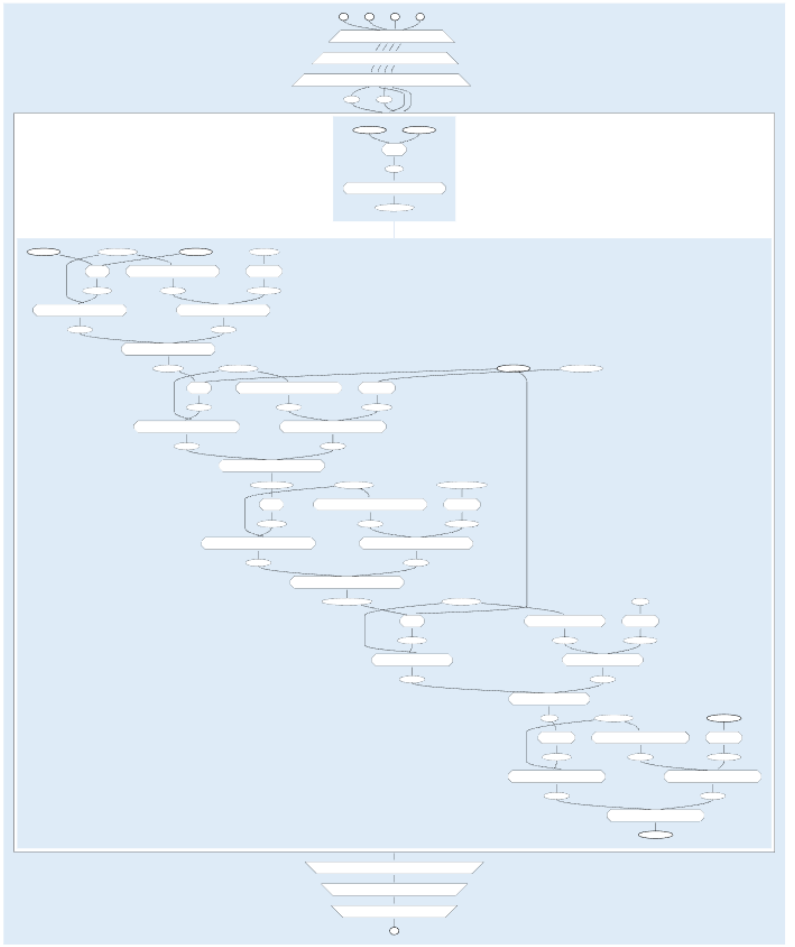
```
1  f_cond = 1.0 if (a[i, j] > 0.5) else 0.0
2  c[i, j] = (a[i, j] * b[i, j]) * f_cond
3            + (0.0) * (1.0 - f_cond)
4  d[i, j] = (1.0 - c[i, j]) * f_cond
5            + (0.0) * (1.0 - f_cond)
```

Explicit Vectorization

- Fixed multiple issues, added support for non-commutative operations, numerically verified the correctness of the pass on multiple representative stencils of CloudSC and element-wise operations.
- Due to the SSA pass, graphs become more dense.

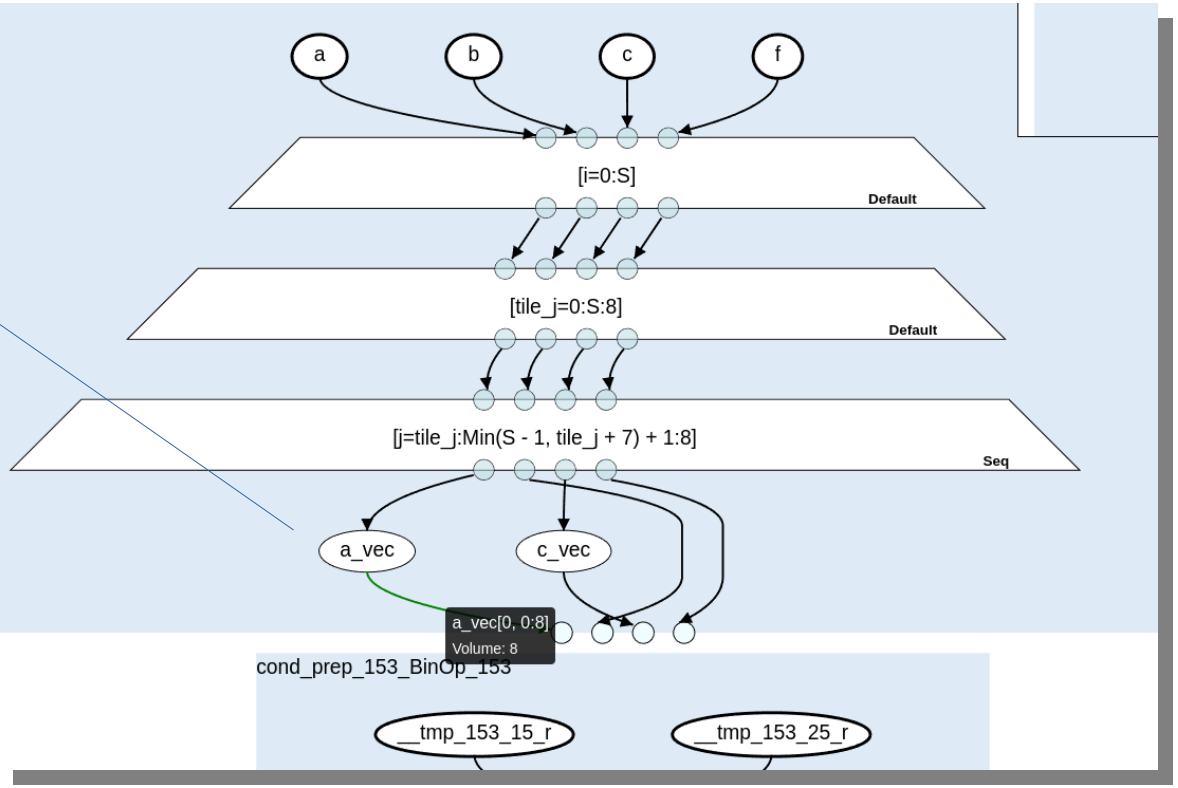


`ExplicitVectorization()`



Optimization of the Data Movement

- Currently data is moved from global (HBM) to fast (L1) memory just before the vector unit requires the memory.
- We plan to implement optimizations to push data movement to earlier stages of the kernel and enable double-buffering support for the SoftHier-backend once we have a numerically verifying baseline



Plan Until Next Meeting

- Obtain vectorized CloudSC SDFG and numerically verify CloudSC on CPU
- Offload CloudSC to SoftHier
- Implement data-movement optimizations and the double buffering optimization for SoftHier